Свёрточные нейронные сети (часть 4)

Свёрточные нейронные сети (часть 4) Лекция 8

Седых И.А.

Минутка кода: размерность тензора

Прежде чем переходить к созданию свёрточного слоя, необходимо создать структуру для тензора. Тензор представляет из себя 3D массив. У тензора имеются три размерности, а значит нам потребуется структура для хранения размера тензора, мы назовём её TensorSize:

```
// размерность тензора
struct TensorSize {
    int depth; // глубина
    int height; // высота
    int width; // ширина
};
```

Минутка кода: класс Тензор - 1

Теперь создадим тензор. В нём нам потребуется хранить depth·height·width вещественных чисел, которые мы будем хранить в одномерном векторе для уменьшения количества обращений к элементам по индексам.

```
// тензор
class Tensor {
   TensorSize size; // размерность тензора
   std::vector<double> values; // значения тензора
   int dw; // произведение глубины на ширину для индексации
   void Init(int width, int height, int depth);

public:
   Tensor(int width, int height, int depth); // создание из размеров
   Tensor(const TensorSize &size); // создание из размера
   double& operator()(int d, int i, int j); // индексация
   double operator()(int d, int i, int j) const; // индексация
   TensorSize GetSize() const; // получение размера
    // вывод тензора
   friend std::ostream& operator<<(std::ostream& os, const Tensor &tensor);
};
```

Минутка кода: класс Тензор - 2

```
// инициализация по размерам
void Tensor::Init(int width, int height, int depth) {
    size.width = width; // запоминаем ширину
    size.height = height; // запоминаем высоту
    size.depth = depth; // запоминаем глубину

    // запоминаем произведение глубины на ширину для индексации
    dw = depth * width;

    values = std::vector<double>(width * height * depth, 0); // создаём вектор из
                                                             // width * height * depth нулей
}

// создание из размеров
Tensor::Tensor(int width, int height, int depth) {
    Init(width, height, depth);
}

// создание из размера
Tensor::Tensor(const TensorSize &size) {
    Init(size.width, size.height, size.depth);
}
```

Минутка кода: класс Тензор - 3

```
// индексация
double& Tensor::operator()(int d, int i, int j) {
    return values[i * dw + j * size.depth + d];
}
double Tensor::operator()(int d, int i, int j) const {
    return values[i * dw + j * size.depth + d];
}
// получение размера
TensorSize Tensor::GetSize() const {
    return size;
}
// вывод тензора
std::ostream& operator<<(std::ostream& os, const Tensor &tensor) {
    for (int d = 0; d < tensor.size.depth; d++) {
        for (int i = 0; i < tensor.size.height; i++) {
            for (int j = 0; j < tensor.size.width; j++)
                os << tensor.values[i * tensor.dw + j * tensor.size.depth + d] << " ";
                os << std::endl;
                }
            os << std::endl;
        }
return os;
}
```

Минутка кода: класс Свёрточный слой - 1

Свёрточный слой содержит в себе набор фильтров и смещений. Также для создания слоя необходимо задать несколько гиперпараметров, : fs — **размер фильтров**, fc — **количество фильтров**, S — **шаг свёртки** и P — **дополнение нулями**. Поскольку фильтры являются тензорами, а смещения всего лишь числами, то добавим фильтры в виде вектора тензоров и назовём W, а смещения добавим в виде векторов вещественных чисел и назовём их b. Поскольку эти веса являются обучаемыми, нам потребуется аналогичные наборы векторов для хранения градиентов.

```
class ConvLayer {
    std::default_random_engine generator;  // генератор случайных чисел
    std::normal_distribution<double> distribution;  // с нормальным распределением

    TensorSize inputSize; // размер входа
    TensorSize outputSize; // размер выхода

    std::vector<Tensor> W; // фильтры
    std::vector<double> b; // смещения

    std::vector<Tensor> dW; // градиенты фильтров
    std::vector<double> db; // градиенты смещений

    int P; // дополнение нулями
    int S; // шаг свёртки
```

Минутка кода: класс Свёрточный слой - 2

```
    int fc; // количество фильтров
    int fs; // размер фильтров
    int fd; // глубина фильтров

void InitWeights(); // инициализация весовых коэффициентов

public:
    ConvLayer(TensorSize size, int fc, int fs, int P, int S); // создание слоя

    Tensor Forward(const Tensor &X); // прямое распространение сигнала

    // обратное распространение ошибки
    Tensor Backward(const Tensor &dout, const Tensor &X);
    void UpdateWeights(double learningRate); // обновление весовых коэффициентов
};
```

Само создание слоя довольно простое: необходимо запомнить переданные параметры в полях класса и инициализировать весовые коэффициенты фильтров и смещений.

Минутка кода: класс Свёрточный слой - 3

```
// создание свёрточного слоя
ConvLayer::ConvLayer(TensorSize size, int fc, int fs, int P, int S) : distribution(0.0, sqrt(2.0 / (fs*fs*size.depth))) {

    // запоминаем входной размер
    inputSize.width = size.width;
    inputSize.height = size.height;
    inputSize.depth = size.depth;

    // вычисляем выходной размер
    outputSize.width = (size.width - fs + 2 * P) / S + 1;
    outputSize.height = (size.height - fs + 2 * P) / S + 1;
    outputSize.depth = fc;

    this->P = P; // сохраняем дополнение нулями
    this->S = S; // сохраняем шаг свёртки

    this->fc = fc; // сохраняем число фильтров
    this->fs = fs; // сохраняем размер фильтров
    this->fd = size.depth; // сохраняем глубину фильтров
```

Минутка кода: класс Свёрточный слой - 4

```
    // добавляем fc тензоров для весов фильтров и их градиентов
    W = std::vector<Tensor>(fc, Tensor(fs, fs, fd));
    dW = std::vector<Tensor>(fc, Tensor(fs, fs, fd));

    // добавляем fc нулей для весов смещения и их градиентов
    b = std::vector<double>(fc, 0);
    db = std::vector<double>(fc, 0);

    InitWeights(); // инициализируем весовые коэффициенты
}
```

Минутка кода: класс Свёрточный слой - 5

Для заполнения весовых коэффициентов случайными числами просто проходимся по всем тензорам и присваиваем каждому элементу очередное случайное число:

```
// инициализация весовых коэффициентов
void ConvLayer::InitWeights() {
   // проходимся по каждому из фильтров
   for (int index = 0; index < fc; index++) {
      for (int i = 0; i < fs; i++)
         for (int j = 0; j < fs; j++)
            for (int k = 0; k < fd; k++)
               // генерируем случайное число и записываем его в элемент фильтра
               W[index](k, i, j) = distribution(generator);

      b[index] = 0.01; // все смещения устанавливаем в 0.01
    }
}
```

Обратное распространение ошибки

Цель обратного распространения ошибки заключается в том, что от следующего слоя нам приходят градиенты некоторой функции потерь L. Исходя из этих градиентов, нам нужно рассчитать градиенты по входному тензору ($\partial L/\partial X$) и градиенты весовых коэффициентов ($\partial L/\partial W$ и $\partial L/\partial b$).

Это становится вполне логичным, если рассматривать нейросеть как **обыкновенную функцию с огромным количеством параметров**, которую мы хотим минимизировать.

Поскольку нейросеть состоит из большого числа блоков, сразу тяжело предрассчитать производные, поэтому на помощь приходит расчёт производной сложной функции $z = g(f(x))$: $z' = g'(f(x)) * f'(x)$.

В нейросети **в качестве функций выступают слои**. Каждый из слоёв умеет вычислять производную по своему аргументу (входному тензору), основываясь на градиентах следующего слоя (или градиентах функции ошибки, если слой является выходным).

Некоторые слои не имеют обучаемых параметров, поэтому они вычисляют только градиенты по своему аргументу.

Свёрточный слой является обучаемым, а потому помимо входных градиентов ему необходимо вычислять и градиенты по весам, чтобы затем их можно было обновить и уменьшить ошибку.

Минутка кода: вычисление градиента по весовым коэффициентам - 1

Вычисление градиента по весовым коэффициентам можно разбить на две части: **по весам фильтров** и **по коэффициентам смещения**.

Проще всего вычисляются градиенты **по весам смещения**: это всего лишь сумма всех градиентов, которые относятся к данному весу смещения:

```
for (int f = 0; f < fc; f++)
    for (int y = 0; y < size.height; y++)
        for (int x = 0; x < size.width; x++)
            db[f] += deltas(f, y, x);
```

Минутка кода: вычисление градиента по весовым коэффициентам - 2

Для вычисления же градиентов **по весам фильтров**, необходимо выполнить немного модифицированную свёртку входного тензора и тензора градиентов:

```
for (int f = 0; f < fc; f++) {
   for (int y = 0; y < size.height; y++) {
      for (int x = 0; x < size.width; x++) {
         double delta = deltas(f, y, x); // запоминаем значение градиента

         for (int i = 0; i < fs; i++) {
            for (int j = 0; j < fs; j++) {
               int i0 = i + y - P;
               int j0 = j + x - P;

               // игнорируем выходящие за границы элементы
               if (i0 < 0 || i0 >= inputSize.height || j0 < 0 || j0 >= inputSize.width)
                  continue;

               // наращиваем градиент фильтра
               for (int c = 0; c < fd; c++)
                  dW[f](c, i, j) += delta * X(c, i0, j0);
            }
         }
      }
   }
}
```

Минутка кода: вычисление градиента по входному тензору - 1

Градиент по входному тензору считается обыкновенной свёрткой тензора дельт и весов с небольшим отличием: для вычисления фильтры поворачиваются на 180 градусов, а дополнение нулями заменяется на величину fs - 1 - P. Данную операцию называют **транспонированной свёрткой** или **кросскорелляцией**.

```
Tensor dX(inputSize);
int pad = fs - 1 - P;
for (int y = 0; y < inputSize.height; y++) {
   for (int x = 0; x < inputSize.width; x++) {
      for (int c = 0; c < fd; c++) {
         double sum = 0; // сумма для градиента
         // идём по всем весовым коэффициентам фильтров
         for (int i = 0; i < fs; i++) {
            for (int j = 0; j < fs; j++) {
               int i0 = y + i - pad;
               int j0 = x + j - pad;
               // игнорируем выходящие за границы элементы
               if (i0 < 0 || i0 >= size.height || j0 < 0 || j0 >= size.width)
                  continue;
               // суммируем по всем фильтрам
               for (int f = 0; f < fc; f++)
                  sum += W[f](c, fs - 1 - i, fs - 1 - j) * deltas(f, i0, j0); // добавляем произведение повёрнутых фильтров
                                                                              //на дельты
            }
         }
         dX(c, y, x) = sum; // записываем результат в тензор градиента
      }
   }
}
```

Минутка кода: вычисление градиента по входному тензору - 2

В заголовке метода Backward использовался параметр dout, а в приведённом выше коде используется параметр deltas, а также неизвестный размер size.

Действительно, deltas и dout не совсем одно и то же. Если шаг свёртки равен 1, то они совпадают, при другом шаге свёртки S deltas вычисляются следующим образом:

```
TensorSize size; // размер дельт

// расчитываем размер для дельт
size.height = S * (outputSize.height - 1) + 1;
size.width = S * (outputSize.width - 1) + 1;
size.depth = outputSize.depth;

Tensor deltas(size); // создаём тензор для дельт

// расчитываем значения дельт
for (int d = 0; d < size.depth; d++)
   for (int i = 0; i < outputSize.height; i++)
      for (int j = 0; j < outputSize.width; j++)
         deltas(d, i * S, j * S) = dout(d, i, j);
```

Минутка кода: обратное распространение - 1

```
// обратное распространение (итоговый код)
Tensor ConvLayer::Backward(const Tensor &dout, const Tensor &X) {
    TensorSize size; // размер дельт

    // расчитываем размер для дельт
    size.height = S * (outputSize.height - 1) + 1;
    size.width = S * (outputSize.width - 1) + 1;
    size.depth = outputSize.depth;

    Tensor deltas(size); // создаём тензор для дельт

    // расчитываем значения дельт
    for (int d = 0; d < size.depth; d++)
        for (int i = 0; i < outputSize.height; i++)
            for (int j = 0; j < outputSize.width; j++)
                deltas(d, i * S, j * S) = dout(d, i, j);
```

Минутка кода: обратное распространение - 2

```
// расчитываем градиенты весов фильтров и смещений
  for (int f = 0; f < fc; f++) {
     for (int y = 0; y < size.height; y++) {
        for (int x = 0; x < size.width; x++) {
           double delta = deltas(f, y, x); // запоминаем значение градиента

           for (int i = 0; i < fs; i++) {
              for (int j = 0; j < fs; j++) {
                 int i0 = i + y - P;
                 int j0 = j + x - P;

                 // игнорируем выходящие за границы элементы
                 if (i0 < 0 || i0 >= inputSize.height || j0 < 0 || j0 >= inputSize.width)
                    continue;

                 // наращиваем градиент фильтра
                 for (int c = 0; c < fd; c++)
                    dW[f](c, i, j) += delta * X(c, i0, j0);
              }
           }

           db[f] += delta; // наращиваем градиент смещения
        }
     }
  }
```

Минутка кода: обратное распространение - 3

```
int pad = fs - 1 - P; // заменяем величину дополнения
 Tensor dX(inputSize); // создаём тензор градиентов по входу

 // рассчитываем значения градиента
 for (int y = 0; y < inputSize.height; y++) {
    for (int x = 0; x < inputSize.width; x++) {
       for (int c = 0; c < fd; c++) {
          double sum = 0; // сумма для градиента
```

Минутка кода: обратное распространение - 4

```
            // идём по всем весовым коэффициентам фильтров
            for (int i = 0; i < fs; i++) {
                for (int j = 0; j < fs; j++) {
                    int i0 = y + i - pad;
                    int j0 = x + j - pad;

                    // игнорируем выходящие за границы элементы
                    if (i0 < 0 || i0 >= size.height || j0 < 0 || j0 >= size.width)
                        continue;

                    // суммируем по всем фильтрам
                    for (int f = 0; f < fc; f++)
                        sum += W[f](c, fs - 1 - i, fs - 1 - j) * deltas(f, i0, j0); // добавляем
                        // произведение повёрнутых фильтров на дельты
                }
            }

            dX(c, y, x) = sum; // записываем результат в тензор градиента
        }
    }
}
return dX; // возвращаем тензор градиентов
}
```

Минутка кода: обновление весовых коэффициентов

После того, как градиенты вычислены, необходимо обновить весовые коэффициенты, чтобы уменьшить ошибку на текущем обучающем примере. Для этого из каждого веса вычитается значение градиента, умноженное на скорость обучения - гиперпараметр метода обычного градиентного спуска.

Данный алгоритм является наиболее простым и довольно медленным с точки зрения сходимости методом оптимизации, можно реализовать более быстрые алгоритмы, как, например, Adam, RMSprop и другие.

```
// обновление весовых коэффициентов
void ConvLayer::UpdateWeights(double learningRate) {
   for (int index = 0; index < fc; index++) {
      for (int i = 0; i < fs; i++) {
         for (int j = 0; j < fs; j++) {
            for (int d = 0; d < fd; d++) {
               W[index](d, i, j) -= learningRate * dW[index](d, i, j); // вычитаем градиент,
               // умноженный на скорость обучения
               dW[index](d, i, j) = 0; // обнуляем градиент фильтра
            }
         }
      }

      b[index] -= learningRate * db[index]; // вычитаем градиент, умноженный на скорость обучения
      db[index] = 0; // обнуляем градиент веса смещения
   }
}
```

Минутка кода: проверка работоспособности - 1

После написания всех методов хотелось бы удостовериться, что они работают корректно. Логичнее всего было бы написать небольшую процедуру для тестирования слоя, однако у нас пока нет возможности задавать свои собственные параметры для весовых коэффициентов. Поэтому создадим два метода для задания значений весовых коэффициентов фильтров и смещений соответственно:

```
// установка веса фильтра по индексу
void ConvLayer::SetWeight(int index, int i, int j, int k, double weight) {
    W[index](i, j, k) = weight;
}

// установка веса смещения по индексу
void ConvLayer::SetBias(int index, double bias) {
    b[index] = bias;
}
```

Нужно далее **самостоятельно** написать небольшую тестирующую функцию, которая проверит значения после применения прямого распространения и обратного распространения.

Пример для проверки - 1

Создадим свёрточный слой, который будет принимать на вход тензор размером 5x5x3 (три канала 5x5), содержать 2 фильтра размером 3x3, шаг смещения 2 и единичное дополнение нулями.

На выходе мы ожидаем тензор размером (5-3+2*1)/2+1 x (5-3+2*1)/2+1 x 2 = 3x3x2.

Возьмём следующие фильтры и входной тензор для проверки:

```
      |-1 1 1 |  |  0 -1  1 |  |  1 -1  0 |
W[0]: |-1 1 1 |  | -1  0 -1 |  | -1  0 -1 |
      | 0 0 1 |  |  1  0  0 |  |  0  1 -1 |

      | 0  0 -1 |  |  1 -1  0 |  |  1  0  1 |
W[1]: | 1  0  0 |  | -1  1  0 |  |  1 -1  1 |
      | 0 -1  0 |  | -1  1  1 |  | -1  0  0 |

b[0]: 1
b[1]: 0

   | 1 2 0 1 0 |  | 1 2 2 1 2 |  | 0 2 0 1 1 |
   | 2 0 0 0 1 |  | 0 2 2 0 2 |  | 2 0 2 1 1 |
X: | 1 2 2 0 2 |  | 1 2 2 1 1 |  | 2 0 1 2 1 |
   | 2 2 2 0 1 |  | 2 2 0 1 0 |  | 0 1 0 1 2 |
   | 2 0 1 0 1 |  | 2 2 1 0 0 |  | 1 0 1 2 1 |
```

На выходе должен получиться следующий тензор:

```
    | 2 -3 1 |  | 3 5  2 |
Y:  | 5 -7 2 |  | 1 0 -3 |
    | 5  0 0 |  |-2 4  3 |
```

Пример для проверки - 2

Пусть теперь слой принимает тензор размером 4x4x1 и содержит 1 фильтр 3x3 с единичным шагом без дополнения нулями:

```
      | 1 4 1 |
W[0]: | 1 4 3 |
      | 3 3 1 |

b[0]: 0

   | 4 5 8 7 |
X: | 1 8 8 8 |
   | 3 6 6 4 |
   | 6 5 7 8 |
```

Результат прямого распространения должен быть таким:

```
Y: | 122 148 |
   | 126 134 |
```

Запустим обратное распространение с такими дельтами:

```
dout: | 2 1 |
      | 4 4 |
```

В результате должен получиться тензор:

```
    |  2  9  6  1 |
dX: |  6 29 30  7 |
    | 10 29 33 13 |
    | 12 24 16  4 |
```

Минутка кода: слой пулинга - 1

```
class MaxPoolingLayer {
    TensorSize inputSize; // размер входа
    TensorSize outputSize; // размер выхода

    int scale; // во сколько раз уменьшается размерность
    Tensor mask; // маска для максимумов

public:
    MaxPoolingLayer(TensorSize size, int scale = 2); // создание слоя

    Tensor Forward(const Tensor &X); // прямое распространение
    Tensor Backward(const Tensor &dout, const Tensor &X); // обратное распространение
};
```

Минутка кода: слой пулинга - 2

Как и свёрточный слой, слой макспулинга создаётся достаточно просто: нужно запомнить входной размер и коэффициент уменьшения размерности, а затем рассчитать размер выходного тензора.

```
MaxPoolingLayer::MaxPoolingLayer(TensorSize size, int scale) : mask(size) {
    // запоминаем входной размер
    inputSize.width = size.width;
    inputSize.height = size.height;
    inputSize.depth = size.depth;

    // вычисляем выходной размер
    outputSize.width = size.width / scale;
    outputSize.height = size.height / scale;
    outputSize.depth = size.depth;

    this->scale = scale; // запоминаем коэффициент уменьшения
}
```

Минутка кода: слой пулинга, прямое распространение - 1

```
// прямое распространение
Tensor MaxPoolingLayer::Forward(const Tensor &X) {
    Tensor output(outputSize); // создаём выходной тензор
    // проходимся по каждому из каналов
    for (int d = 0; d < inputSize.depth; d++) {
        for (int i = 0; i < inputSize.height; i += scale) {
            for (int j = 0; j < inputSize.width; j += scale) {
                int imax = i; // индекс строки максимума
                int jmax = j; // индекс столбца максимума
                double max = X(d, i, j); // начальное значение максимума –
                //значение первой клетки подматрицы

                // проходимся по подматрице и ищем максимум и его координаты
                for (int y = i; y < i + scale; y++) {
                    for (int x = j; x < j + scale; x++) {
                        double value = X(d, y, x); // получаем значение входного тензора
                        mask(d, y, x) = 0; // обнуляем маску

                        // если очередное входное значение больше максимального
                        if (value > max) {
                            imax = y; // обновляем индекс строки максимума
                            jmax = x; // обновляем индекс столбца максимума
                            max = value; // обновляем максимум
                        }
                    }
                }
```

Минутка кода: слой пулинга, прямое распространение - 2

```
            // записываем в выходной тензор найденный максимум
            output(d, i / scale, j / scale) = max;

            // записываем 1 в маску в месте расположения максимального элемента
            mask(d, imax, jmax) = 1;
        }
    }
  }
  return output; // возвращаем выходной тензор
}
```

Минутка кода: слой пулинга, обратное распространение

Обратное распространение ошибки для слоя макспулинга сводится к умножению градиентов следующего слоя на рассчитанную маску.

```
// обратное распространение
Tensor MaxPoolingLayer::Backward(const Tensor &dout, const Tensor &X) {
   Tensor dX(inputSize); // создаём тензор для градиентов

   for (int d = 0; d < inputSize.depth; d++)
      for (int i = 0; i < inputSize.height; i++)
         for (int j = 0; j < inputSize.width; j++)
            dX(d, i, j) = dout(d, i / scale, j / scale) * mask(d, i, j); // умножаем градиенты на маску

   return dX; // возвращаем посчитанные градиенты
}
```

Заметим, что для **среднего пулинга** и **пулинга суммы** не требуется добавление бинарной маски, так как в них при обратном распространении градиенты следующего слоя просто копируются во все элементы соответствующих подматриц (для среднего градиенты делятся на общее количество элементов подматрицы).

Минутка кода: слой активации

Активационный слой не изменяет размерность входного тензора, а потому вместо хранения двух размеров, мы будем хранить лишь один.

В качестве примера продемонстрируем слой активации с линейным выпрямитем (ReLU).

```
class ReLULayer {
   TensorSize size; // размер слоя

public:
   ReLULayer(TensorSize size); // создание слоя

   Tensor Forward(const Tensor &X); // прямое распространение
   Tensor Backward(const Tensor &dout, const Tensor &X); // обратное распространение
};

// создание слоя
ReLULayer::ReLULayer(TensorSize size) {
   this->size = size; // сохраняем размер
}
```

Минутка кода: слой активации, прямое распространение

На этапе прямого распространения необходимо пройтись по каждому элементу входного тензора и применить активационную функцию. В данном случае функцией является ReLU = max(0, x).

```
// прямое распространение
Tensor ReLULayer::Forward(const Tensor &X) {
    Tensor output(size); // создаём выходной тензор

    // проходимся по всем значениям входного тензора
    for (int i = 0; i < size.height; i++)
        for (int j = 0; j < size.width; j++)
            for (int k = 0; k < size.depth; k++)
                output(k, i, j) = X(k, i, j) > 0 ? X(k, i, j) : 0; // вычисляем значение функции активации

    return output; // возвращаем выходной тензор
}
```

Минутка кода: слой активации, обратное распространение

Так как каждое значение тензора было преобразовано с помощью одной и той же функции, то для получения градиентов по входу нужно просто умножить градиенты следующего слоя на значения производных функции активации.

```
Tensor ReLULayer::Backward(const Tensor &dout, const Tensor &X) {
    Tensor dX(size); // создаём тензор градиентов

    // проходимся по всем значениям тензора градиентов
    for (int i = 0; i < size.height; i++)
        for (int j = 0; j < size.width; j++)
            for (int k = 0; k < size.depth; k++)
                dX(k, i, j) = dout(k, i, j) * (X(k, i, j) > 0 ? 1 : 0); // умножаем градиенты
следующего слоя
                                                                        //на производную функции
активации

    return dX; // возвращаем тензор градиентов
}
```

Минутка кода: полносвязный слой, класс матрица - 1

Полносвязный слой содержит матрицу весовых коэффициентов. В качестве матрицы можно было бы использовать тензор с глубиной равной 1, но для большей наглядности мы создадим отдельный класс для неё. В классе будет два числа — число строк и число столбцов матрицы, а также вектор векторов для хранения самих значений. Для получения и изменения элементов матрицы понадобится индексация — перегрузим для этого оператор круглые скобки.

```
class Matrix {
    int rows; // число строк
    int columns; // число столбцов
    std::vector<std::vector<double>> values; // значения

public:
    Matrix(int rows, int columns); // конструктор из заданных размеров

    double& operator()(int i, int j); // индексация
    double operator()(int i, int j) const; // индексация
};
```

Минутка кода: полносвязный слой, класс матрица - 2

```
// конструктор из заданных размеров
Matrix::Matrix(int rows, int columns) {
    this->rows = rows; // сохраняем число строк
    this->columns = columns; // сохраняем число столбцов

    // создаём векторы для значений матрицы
    values = std::vector<std::vector<double>>(rows, std::vector<double>(columns, 0));
}

// индексация
double& Matrix::operator()(int i, int j) {
    return values[i][j];
}

// индексация
double Matrix::operator()(int i, int j) const {
    return values[i][j];
}
```

Минутка кода: класс полносвязный слой - 1

В классе нам потребуется хранить матрицу весовых коэффициентов и вектор весов смещений. Поскольку слой содержит обучаемые параметры, их нужно будет инициализировать, так что добавим генератор случайных чисел с нормальным распределением.

Помимо этого нужно добавить возможность применения активационной функции, а потому потребуется знать, какую функцию активации применять. Для этого создадим перечисление из основных функций активации и при создании слоя будем передавать имя активационной функции через строковый параметр.

Для обратного распространения ошибки необходимо будет вычислять производные активационных функций, а потому добавим тензор df, который будет хранить эти значения.

Для применения функции активации напишем метод Activate, который также будет заполнять значения производной выбранной функции.

Минутка кода: класс полносвязный слой - 2

```
class FullyConnectedLayer {
    // тип активационной функции
    enum class ActivationType {
        None, // без активации
        Sigmoid, // сигмоидальная функция
        Tanh, // гиперболический тангенс
        ReLU, // выпрямитель
        LeakyReLU, // выпрямитель с утечкой
        ELU // экспоненциальный выпрямитель
    };

    TensorSize inputSize; // входной размер
    TensorSize outputSize; // выходной размер

    std::default_random_engine generator;
    std::normal_distribution<double> distribution;

    int inputs; // число входных нейронов
    int outputs; // число выходных нейронов
```

Минутка кода: класс полносвязный слой - 3

```
ActivationType activationType; // тип активационной функции
    Tensor df; // тензор производных функции активации

    Matrix W; // матрица весовых коэффициентов
    Matrix dW; // матрица градиентов весовых коэффициентов

    std::vector<double> b; // смещения
    std::vector<double> db; // градиенты смещений

     // получение типа активационной функции по строке
    ActivationType GetActivationType(const std::string& activationType) const;

    void InitWeights(); // инициализация весовых коэффициентов
    void Activate(Tensor &output); // применение активационной функции

public:
     // создание слоя
    FullyConnectedLayer(TensorSize size, int outputs, const std::string& activationType = "none");

    Tensor Forward(const Tensor &X); // прямое распространение
    Tensor Backward(const Tensor &dout, const Tensor &X); // обратное распространение

    void UpdateWeights(double learningRate); // обновление весовых коэффициентов
};
```

Минутка кода: класс полносвязный слой - 4

Слой создаётся из размера входного тензора, количества выходных нейронов и типа функции активации. Внутри конструктора слоя необходимо запомнить входной и выходной размеры, получить тип активационной функции, а также проинициализировать весовые коэффициенты.

```
FullyConnectedLayer::FullyConnectedLayer(TensorSize size, int outputs, const std::string&
activationType) : W(outputs, size.height * size.width * size.deep), dW(outputs, size.height * size.width
* size.deep), df(1, 1, outputs), distribution(0.0, sqrt(2.0 / (size.height * size.width * size.deep))) {
    // запоминаем входной размер
    inputSize.width = size.width;
    inputSize.height = size.height;
    inputSize.depth = size.depth;

    // вычисляем выходной размер
    outputSize.width = 1;
    outputSize.height = 1;
    outputSize.depth = outputs;

    this->inputs = size.height * size.width * size.deep; // запоминаем число входных нейронов
    this->outputs = outputs; // запоминаем число выходных нейронов

    this->activationType = GetActivationType(activationType); // получаем активационную функцию

    b = std::vector<double>(outputs); // создаём вектор смещений
    db = std::vector<double>(outputs); // создаём вектор градиентов по весам смещения

    InitWeights(); // инициализируем весовые коэффициенты
}
```

Минутка кода: класс полносвязный слой - 5

Так как тип функции активации передается в виде строки, а внутри класса работаем со значением перечисления ActivationType, нам необходимо описать метод, получающий значение перечисления по введённой строке. В нём также нужно будет проверить, что введённая строка действительно описывает одну из функций активации, а не какой-нибудь мусор. Для этого мы будем бросать исключение, если ни одна из ожидаемых функций активации не подошла.

```
// получение типа активационной функции по строке
FullyConnectedLayer::ActivationType FullyConnectedLayer::GetActivationType(const std::string&
activationType) const {
    if (activationType == "sigmoid")
        return ActivationType::Sigmoid;
    if (activationType == "tanh")
        return ActivationType::Tanh;
    if (activationType == "relu")
        return ActivationType::ReLU;
    if (activationType == "leakyrelu")
        return ActivationType::LeakyReLU;
    if (activationType == "elu")
        return ActivationType::ELU;
    if (activationType == "none" || activationType == "")
        return ActivationType::None;

    throw std::runtime_error("Invalid activation function");
}
```

Минутка кода: полносвязный слой, инициализация весовых коэффициентов

```
// инициализация весовых коэффициентов
void FullyConnectedLayer::InitWeights() {
    for (int i = 0; i < outputs; i++) {
        for (int j = 0; j < inputs; j++)
            W(i, j) = distribution(generator); // генерируем очередное случайное число

        b[i] = 0.01; // все смещения делаем равными 0.01
    }
}
```

Минутка кода: полносвязный слой, функция активации и ее производная - 1

В прямом распространении нужно умножить матрицу весовых коэффициентов на входной тензор, представленный в виде **одномерного вектора**, прибавить вес смещения, а затем применить активационную функцию.

```
// применение активационной функции с вычислением значений её производной
void FullyConnectedLayer::Activate(Tensor &output) {
    if (activationType == ActivationType::None) {
        for (int i = 0; i < outputs; i++) {
            df[i] = 1;
        }
    }
    else if (activationType == ActivationType::Sigmoid) {
        for (int i = 0; i < outputs; i++) {
            output[i] = 1 / (1 + exp(-output[i]));
            df[i] = output[i] * (1 - output[i]);
        }
    }
    else if (activationType == ActivationType::Tanh) {
        for (int i = 0; i < outputs; i++) {
            output[i] = tanh(output[i]);
            df[i] = 1 - output[i] * output[i];
        }
    }
```

Минутка кода: полносвязный слой, функция активации и ее производная - 2

```
else if (activationType == ActivationType::ReLU) {
        for (int i = 0; i < outputs; i++) {
            if (output[i] > 0) {
                df[i] = 1;
            }
            else {
                output[i] = 0;
                df[i] = 0;
            }
        }
    }
    else if (activationType == ActivationType::LeakyReLU) {
        for (int i = 0; i < outputs; i++) {
            if (output[i] > 0) {
                df[i] = 1;
            }
            else {
                output[i] *= 0.01;
                df[i] = 0.01;
            }
        }
    }
```

Минутка кода: полносвязный слой, функция активации и ее производная - 3

```
else if (activationType == ActivationType::ELU) {
      for (int i = 0; i < outputs; i++) {
         if (value > 0) {
            df[i] = 1;
         }
         else {
            output[i] = exp(output[i]) - 1;
            df[i] = output[i] + 1;
         }
      }
   }
}
```

Минутка кода: полносвязный слой, прямое распространение

Чтобы умножить матрицу на входной вектор, нужно в каждой строке матрицы пройтись по всем её элементам и умножить их на соответствующие элементы входного вектора X.

Для получения выходного значения к получаемому в каждой строке числу нужно прибавить вес смещения и записать в выходной тензор результат активационной функции.

В целях оптимизации мы будем сначала записывать значения произведения матрицы на вектор с добавленным весом смещения, потом проходиться по всем полученным значениям и применять активационную функцию.

```
// прямое распространение
void FullyConnectedLayer::Forward(const Tensor &X) {
   Tensor output(outputSize); // создаём выходной тензор

   // проходимся по каждому выходному нейрону
   for (int i = 0; i < outputs; i++) {
      double sum = b[i]; // прибавляем смещение

      // умножаем входной тензор на матрицу
      for (int j = 0; j < inputs; j++)
         sum += W(i, j) * X[j];

      output[i] = sum;
   }

   Activate(output); // применяем активационную функцию

   return output; // возвращаем выходной тензор
}
```

Минутка кода: полносвязный слой, обратное распространение - 1

Обратное распространение ошибки для полносвязного слоя можно разделить на **два этапа**: вычисление градиентов по весовым коэфициентам, которое делится на вычисление градиентов смещений и градиентов матрицы, и вычисление градиентов по входам:

- **градиентами по весам смещения** являются произведения градиентов следующего слоя на значение производной функции активации: db = dout*df;

- для вычисления **градиентов матрицы весов** нужно умножить градиенты следующего слоя и значения производной активационной функции на входные значения: dW = (dout*df)·X;

- для вычисления **градиентов по входам** нужно умножить градиенты следующего слоя и значения производной активационной функции на транспонированную матрицу весовых коэффициентов: $dX = W^T \cdot (dout*df)$.

Поскольку постоянно перемножать выходные градиенты на производные не очень эффективно, перед вычислением градиентов домножим тензор производных на тензор градиентов следующего слоя.

Минутка кода: полносвязный слой, обратное распространение - 2

```
// обратное распространение
Tensor FullyConnectedLayer::Backward(const Tensor &dout, const Tensor &X) {
   // домножаем производные на градиенты следующего слоя
   // для сокращения количества умножений
   for (int i = 0; i < outputs; i++)
      df[i] *= dout[i];

   // вычисляем градиенты по весовым коэффициентам
   for (int i = 0; i < outputs; i++) {
      for (int j = 0; j < inputs; j++)
         dW(i, j) = df[i] * X[j];

      db[i] = df[i];
   }

   Tensor dX(inputSize); // создаём тензор для градиентов по входам

   // вычисляем градиенты по входам
   for (int j = 0; j < inputs; j++) {
      double sum = 0;

      for (int i = 0; i < outputs; i++)
         sum += W(i, j) * df[i];

      dX[j] = sum; // записываем результат в тензор градиентов
   }

   return dX; // возвращаем тензор градиентов
}
```

Минутка кода: полносвязный слой, обновление весовых коэффициентов

Обновление весовых коэффициентов полносвязного слоя ничем не отличается от обновления параметров свёрточного слоя: из каждого весового коэффициента нужно вычесть значение градиента, умноженное на скорость обучения.

```
// обновление весовых коэффициентов
void FullyConnectedLayer::UpdateWeights(double learningRate) {
   for (int i = 0; i < outputs; i++) {
      for (int j = 0; j < inputs; j++)
         W(i, j) -= learningRate * dW(i, j);

      b[i] -= learningRate * db[i]; // обновляем веса смещения
   }
}
```

Минутка кода: полносвязный слой, тестирование - 1

Чтобы убедиться, что слой создан корректно, мы проверим его работу с определёнными весами. Сейчас у нас нет возможности задать определённые весовые коэффициенты слою, поэтому добавим два метода для установки весов матрицы и весов смещения.

```
// установка веса матрицы
void FullyConnectedLayer::SetWeight(int i, int j, double weight) {
    W(i, j) = weight;
}

// установка веса смешения
void FullyConnectedLayer::SetBias(int i, double bias) {
    b[i] = bias;
}
```

Минутка кода: полносвязный слой, тестирование - 2

После задания весов слою проверим работу прямого и обратного распространения. Для проверки создадим полносвязный слой, принимающий на входе тензор размером 1x1x8, и состоящий из 4 нейронов и использующий активационную функцию ReLU. Таким образом размер выходного тензора будет 1x1x4.

```
   | 1  2  3  4 -1 -2 -3 -4 |
W: | 1  0  0  1 -1  2 -3  4 |
   | 1  1  1  1 -3 -4 -5 -6 |
   | 1  2  2  1  2  2 -3 -8 |

b: | 0 0 0 0 |

X: | 1 2 -3 4 0 -7 2 -4 |
```

Если всё сделано правильно, то на выходе должен получиться следующий тензор:

```
Y: | 36 0 46 15 |
```

Для обратного распространения зададим следующие градиенты:

```
dout: | -0.5 0.1 -0.25 0.7 |
```

Выполнив обратное распространение ошибки слой должен вернуть такой тензор:

```
dX: | -0.05 0.15 -0.35 -1.55 2.65 3.4 0.65 -2.1 |
```